DEBUGGING · DOMAIN 9 - DEBUGGING AND DEPLOYING

Job repair and parameter overrides

Section 9. Rerunning the four tasks that failed, not the eleven that worked.

SECTION 9 of 10 WEIGHT 10% TOPIC Repair

admin@sthithapragna.com

A repair is part of the original run, not a new one. It reruns only what failed, and it picks up your fix.

What repair actually does

- Re-runs **only the failed tasks**, and any dependent tasks that need to follow them.
- Everything that already succeeded is left alone, which is the whole point on a long job.
- **The run must not be in progress.** You cannot repair one failed task while others are still running.
- Contrast with a restart, which re-runs the entire job from the beginning.

> ON THE EXAM
>
> The 'not in progress' rule surprises people managing large fan-out jobs. You wait for the run to settle, then repair.

A repair is not a replay

- Repaired tasks use the **current job and task settings**, not the settings as they were at the original run.
- So the sequence is: diagnose, **fix the job definition**, then repair. The repair runs the fixed version.
- That also means a repair days later may behave differently from the original run, if somebody edited the job.
- The repair still appears in the **history of the original run**, keeping the record together.

> ON THE EXAM
>
> This is why repair is a remediation mechanism. A pure replay of the failed configuration would just fail again.

What you can ask a repair to do

OPTION	EFFECT
Rerun named tasks	Repair specific task keys that you choose.
Rerun all failed tasks	Repair everything that failed. Cannot be combined with naming tasks.
Rerun dependent tasks	Also repair everything downstream, even if it previously succeeded.
Job parameters	Override job-level parameters for this repair.
Performance target	Override the job's serverless performance target for this repair.
Full refresh	Trigger a full refresh on a declarative pipeline task.

ON THE EXAM

Naming tasks and rerunning all failed tasks are mutually exclusive. You pick one or the other.

Why rerunning dependents matters

- By default a repair reruns the failed tasks and the dependents that were **skipped** because of them.
- The dependent option goes further, rerunning downstream tasks **even if they previously succeeded**.
- You need that when a failed task **partially wrote data** that a successful downstream task then consumed.
- Otherwise the downstream result is built on data that has now changed underneath it.

> ON THE EXAM
>
> A task succeeding does not mean its output is still correct. If an upstream input changed on repair, the downstream needs redoing.

Disabled tasks and repair

- **Repairs use each task's run state to decide what to repair, not its disabled state.**
- So a task disabled after the original run may still be selected for repair based on how it ran.
- To **force** a disabled task to run during a repair, name it explicitly in the rerun list.
- Precise enough to be a question, and unlikely to be guessed correctly.

> ON THE EXAM
>
> Run state and disabled state are two different things, and repair looks at the first one.

Sequential repairs need the previous repair ID

- The first repair of a run needs no repair identifier.
- **Every subsequent repair of the same run must supply the identifier of the latest repair.**
- That is what keeps repairs sequential rather than conflicting.
- So a script that repairs automatically has to capture the returned identifier and pass it on next time.

> ON THE EXAM
>
> An easy thing to get wrong when automating. The first call works, and the second one fails without an obvious reason.

Overriding values for one repair

- The repair dialog lists the unsuccessful tasks **and the dependent tasks that will be re-run**.
- Parameters you enter there **override the existing values**, for that repair only.
- On a later repair, **clearing the key and value returns the parameter to its original value**.
- Overrides do not change the job definition. A scheduled run tomorrow uses the original values.

> ON THE EXAM
>
> That last point is the safety property. You can rerun with a corrected date without permanently editing the job.

Duration includes every repair

- The duration shown in the runs tab spans from **the first run's start to the latest repair's finish**.
- A run that failed twice and succeeded on the third attempt reports the time for **all three**.
- So a repaired run can trip a **run duration health threshold** even though the successful attempt was quick.
- That is module 19's duration warning notification, firing for a reason nobody expected.

> ON THE EXAM
>
> Worth knowing before somebody investigates a slow job that was never slow. It failed, waited, and was repaired.

Failed and skipped look similar

- The **matrix view** in the runs tab shows successful and unsuccessful runs for every task across many runs.
- A task run may be unsuccessful because it **failed**, or because it was **skipped when a dependency failed**.
- Those mean completely different things, and only one of them points at a problem in that task.
- Hovering over a failed task shows its metadata, which is how you tell them apart quickly.

> ON THE EXAM
>
> Chasing a task that was merely skipped is a common waste of time. The failure is upstream of it.

Full refresh is available, and dangerous

- A repair can trigger a **full refresh** on a declarative pipeline task.
- That clears the streaming state and reprocesses everything, exactly as in module 5.
- **Any source beyond its retention period will not be reprocessed**, and that data is lost.
- So reach for it when the pipeline logic changed, not merely because a run failed.

> ON THE EXAM
>
> Full refresh as a repair option is convenient and it is the same operation with the same consequences. Retention arithmetic first.

Is the task safe to run again?

- A repair reruns a task that may have **partially completed** before it failed.
- If the task is **idempotent**, rerunning is safe. A merge on a key, or an overwrite, is fine.
- If it **appends** without deduplication, or calls an external system, rerunning may double the effect.
- This is module 9's at-most-once problem, arriving from a different direction.

> ON THE EXAM
>
> Repair is safe when your tasks are idempotent. Designing for that is what makes recovery routine rather than an incident.

Exceeding maximum concurrent runs

- A run can fail simply because the job's **maximum concurrent runs** was exceeded.
- The documented remedies: **wait for other runs to complete**, or **increase the limit**.
- Worth recognising because the failure has nothing to do with your code or your data.
- Repairing without addressing it will simply fail the same way.

> ON THE EXAM
>
> A failure that no amount of log reading will explain, because nothing went wrong inside the job at all.

What to carry into the exam

- A repair reruns **only failed tasks and dependents**, as part of the **original run**, using **current settings**.
- The run **must not be in progress**. Naming tasks and rerunning all failed tasks are **mutually exclusive**.
- Subsequent repairs of the same run need the **latest repair identifier**.
- Parameter overrides apply **to that repair only**. Clearing them restores the original value.
- **Duration spans every attempt**, so a repaired run can trip a duration alert.

> ON THE EXAM
>
> When a question describes a long job failing near the end, repair is almost always the intended answer. The detail decides which option is right.

> A fifteen-task job fails at task twelve because a configuration value was wrong. The engineer corrects the value in the job definition, then wants to complete the run without repeating the eleven tasks that succeeded.
>
> Those eleven tasks took four hours.

What should they do?

A. Restart the job, since the definition changed and the new value must be picked up.

B. Repair the run, which reruns only the failed and dependent tasks using the current settings.

C. Clone the job, delete the first eleven tasks, and run the clone.

D. Repair the run, but the old configuration value will be used because repairs replay the original settings.

◆ PAUSE AND DECIDE

What should they do?

A Restart the job, since the definition changed and the new value must be picked up.

Repeats four hours of successful work. Repair also picks up the new value.

B Repair the run, which reruns only the failed and dependent tasks using the current settings.

Repaired tasks use current job and task settings, so the corrected value is applied without repeating the successful tasks.

C Clone the job, delete the first eleven tasks, and run the clone.

Creates a second job to maintain, and loses the run history connection.

D Repair the run, but the old configuration value will be used because repairs replay the original settings.

The opposite of documented behaviour. Repairs use current settings.

ON THE EXAM

D is the important distractor. If repairs replayed the original configuration, they would be useless for fixing configuration errors.

> A job's task C failed after writing roughly half its output. Task D depends on C, ran anyway on a schedule quirk, and completed successfully against that partial data. Task E depends on D and was skipped.
>
> The engineer plans to repair, selecting only task C.

What is the problem?

A. Nothing. Repairing C will automatically re-run D and E.

B. D succeeded against partial data, so it will not be re-run by default. Use the option to rerun dependent tasks.

C. Task C cannot be repaired because it partially wrote output.

D. The whole job must be restarted, since repair cannot handle partial writes.

◆ PAUSE AND DECIDE

What is the problem?

A Nothing. Repairing C will automatically re-run D and E.

E was skipped so it will re-run. D succeeded, so by default it will not.

B D succeeded against partial data, so it will not be re-run by default. Use the option to rerun dependent tasks.

That option re-runs downstream tasks even where they previously succeeded, which is exactly this case.

C Task C cannot be repaired because it partially wrote output.

It can. Whether that is safe depends on whether C is idempotent.

D The whole job must be restarted, since repair cannot handle partial writes.

Repair handles this with the dependent option, without repeating everything.

ON THE EXAM

Succeeded is not the same as still correct. D's output was computed from data that is about to change underneath it.

> A team automates recovery with a script that calls the repair API. The first repair of a failed run succeeds. When the repaired run fails again and the script repairs a second time, the API rejects the request.
>
> The script sends the same request body each time, with the run ID and the failed task keys.

What is missing?

A. The latest repair identifier, which is required on every repair after the first.

B. A run cannot be repaired more than once; the script must restart the job instead.

C. The script must wait 24 hours between repairs of the same run.

D. Both rerun_tasks and rerun_all_failed_tasks must be supplied together on subsequent repairs.

◆ PAUSE AND DECIDE

What is missing?

A The latest repair identifier, which is required on every repair after the first.

Not required for the first repair, mandatory on subsequent ones, to keep repairs sequential.

B A run cannot be repaired more than once; the script must restart the job instead.

Runs can be repaired repeatedly, which is why the sequencing identifier exists.

C The script must wait 24 hours between repairs of the same run.

Invented.

D Both rerun_tasks and rerun_all_failed_tasks must be supplied together on subsequent repairs.

They are mutually exclusive; only one may be used.

ON THE EXAM

The symptom is diagnostic: works the first time, fails the second. That pattern points at something required only on repeat calls.

> A daily job normally runs in eight minutes. One morning it fails, and an engineer repairs it four hours later. The repair completes in six minutes.
>
> A duration warning notification fires, reporting the run exceeded its threshold. The team investigates for performance problems and finds none.

What explains the alert?

A. Duration spans from the first run's start to the latest repair's finish, so it includes the four idle hours.

B. The repair ran on smaller compute, making it slower than reported.

C. Duration warnings fire on any repaired run regardless of timing.

D. The threshold was misconfigured and should be raised.

◆ PAUSE AND DECIDE

What explains the alert?

A. Duration spans from the first run's start to the latest repair's finish, so it includes the four idle hours.

Documented: duration runs from the first start to the latest repair's finish, covering every attempt and the gaps.

B. The repair ran on smaller compute, making it slower than reported.

The repair is stated to have completed in six minutes.

C. Duration warnings fire on any repaired run regardless of timing.

They fire on the threshold, which this run genuinely exceeded as measured.

D. The threshold was misconfigured and should be raised.

Raising it would hide genuinely slow runs to accommodate an artefact of repair timing.

ON THE EXAM

The alert is correct by its own definition. The team's mistake is assuming duration means the time spent computing.

> A job ingests a file whose path is built from a date parameter. One night the upstream system delivered the file a day late, so the run failed on a missing path. The file is now present under yesterday's date.
>
> The team wants to complete that run without changing tomorrow's scheduled behaviour.

What should they do?

A. Edit the job's default date parameter, repair the run, then edit it back afterwards.

B. Repair the run, entering the corrected date as a parameter override in the repair dialog.

C. Create a one-off job with the corrected date and run that instead.

D. Restart the job, which prompts for parameters at launch.

◆ PAUSE AND DECIDE

What should they do?

A Edit the job's default date parameter, repair the run, then edit it back afterwards.

Works if nobody forgets the second edit, and mutates a production job to fix one run.

B Repair the run, entering the corrected date as a parameter override in the repair dialog.

Overrides apply to that repair only, leaving the job definition and tomorrow's schedule untouched.

C Create a one-off job with the corrected date and run that instead.

Leaves the original run failed in the history, and creates a second job.

D Restart the job, which prompts for parameters at launch.

Repeats every task, and this run only needs the failed one.

ON THE EXAM

The safety property is that overrides are scoped to the repair. The job definition, and therefore tomorrow's run, is untouched.

> A job task calls an external payment provider's API to issue refunds, then records them in a table. It failed partway through, after issuing some refunds but before recording them all.
>
> An engineer is about to repair the run.

What should be checked first?

A Nothing. Repair reruns only what failed, so already-issued refunds will not be reissued.

B Whether the task is idempotent. A repair reruns a task that partially completed, and refunds are not idempotent.

C Whether the run is still in progress, which would prevent repair.

D Whether the latest repair identifier is available.

◆ PAUSE AND DECIDE

What should be checked first?

A Nothing. Repair reruns only what failed, so already-issued refunds will not be reissued.

The failed task is re-run in full. Work it completed before failing is repeated.

B Whether the task is idempotent. A repair reruns a task that partially completed, and refunds are not idempotent.

Repair re-runs the whole task, so a non-idempotent task repeats side effects it already performed.

C Whether the run is still in progress, which would prevent repair.

A real precondition, and a mechanical one rather than the risk here.

D Whether the latest repair identifier is available.

Relevant for a second repair, and not the question before the first.

ON THE EXAM

Repair reruns the task, not the part of it that failed. That distinction is what makes idempotency the first question.